

Backpropagation Example on a Neural Network (Optional)

This lesson walks through a concrete example of forward and backward propagation on a simple neural network to build intuition for how the backpropagation algorithm works and why it is so efficient.

The Example Network & Setup

- **Architecture:** A simple 2-layer network.
 - 1 hidden layer with 1 neuron (a_1).
 - 1 output layer with 1 neuron (a_2).
 - **Activation:** ReLU ($g(z) = \max(0, z)$) for all layers.
 - **Cost Function:** Squared Error, $J = \frac{1}{2}(a_2 - y)^2$.
 - **Training Example:** $x = 1, y = 5$.
 - **Parameters:**
 - Layer 1: $w_1 = 2, b_1 = 0$
 - Layer 2: $w_2 = 3, b_2 = 1$
-

1. Forward Propagation (Left-to-Right →)

We first calculate the output and the final cost by moving forward through the network.

1. Layer 1:

- $z_1 = w_1x + b_1 = (2 \times 1) + 0 = 2$
- $a_1 = g(z_1) = \max(0, 2) = 2$

2. Layer 2:

- $z_2 = w_2a_1 + b_2 = (3 \times 2) + 1 = 7$
- $a_2 = g(z_2) = \max(0, 7) = 7$

3. Cost:

- $J = \frac{1}{2}(a_2 - y)^2 = \frac{1}{2}(7 - 5)^2 = \frac{1}{2}(2)^2 = 2$

This entire process can be mapped to a **computation graph** showing each step.

2. Backpropagation (Right-to-Left ←)

Backpropagation is the process of moving backward through the graph (from right to left) to calculate the derivative of the final cost J with respect to every parameter.

Without showing every intermediate step (which was covered in the previous video), this process would find the derivatives for all four parameters:

- $\frac{\partial J}{\partial w_1} = 6$
- $\frac{\partial J}{\partial b_1} = 2$
- $\frac{\partial J}{\partial w_2} = 2$
- $\frac{\partial J}{\partial b_2} = 2$

The key result we will verify is $\frac{\partial J}{\partial w_1} = 6$.

3. Verifying the Derivative (The "Nudge" Check) 🧱

We can confirm the result of backprop by manually "nudging" a parameter and re-running forward propagation to see the effect on the final cost J .

- **Goal:** Check if $\frac{\partial J}{\partial w_1} = 6$ is correct.
 - **Original values:** $w_1 = 2$, $J = 2$.
 - **Nudge w_1 :** Let's increase w_1 by a tiny amount, $\epsilon = 0.001$.
 - $w_1^{\text{new}} = 2.001$
 - **Recalculate Forward Prop:**
 1. **Layer 1:**
 - $z_1 = (2.001 \times 1) + 0 = 2.001$
 - $a_1 = \max(0, 2.001) = 2.001$
 2. **Layer 2:**
 - $z_2 = (3 \times 2.001) + 1 = 6.003 + 1 = 7.003$
 - $a_2 = \max(0, 7.003) = 7.003$
 3. **Cost:**
 - $J^{\text{new}} = \frac{1}{2}(7.003 - 5)^2 = \frac{1}{2}(2.003)^2 \approx 2.006005$
 - **Analyze the Change:**
 - We increased w_1 by 0.001 (ϵ).
 - The cost J increased from 2.000 to ≈ 2.006 .
 - The change in J is ≈ 0.006 , which is **6 times** the change in w_1 .
 - **Conclusion:** This confirms that the derivative $\frac{\partial J}{\partial w_1}$ is indeed **6**.
-

Why Backpropagation is Efficient ⚡

- **The "Nudge" Method (Numerical Differentiation):** The check we just performed is a valid way to find a derivative. However, to find all 4 derivatives, we would have to run this **entire forward propagation process 4 times** (once for each parameter).
 - For P parameters and N nodes, this takes $O(N \times P)$ steps. This is **very slow** for large networks.
- **Backpropagation (Automatic Differentiation):**
 - Backprop is a clever algorithm that moves right-to-left, **reusing calculations** from later layers to compute derivatives for earlier layers.
 - It can compute *all* derivatives in roughly $O(N + P)$ steps, which is **vastly more efficient**.
 - This is the "magic" that frameworks like TensorFlow use. The process is called **automatic differentiation (autodiff)**.